DuocUC

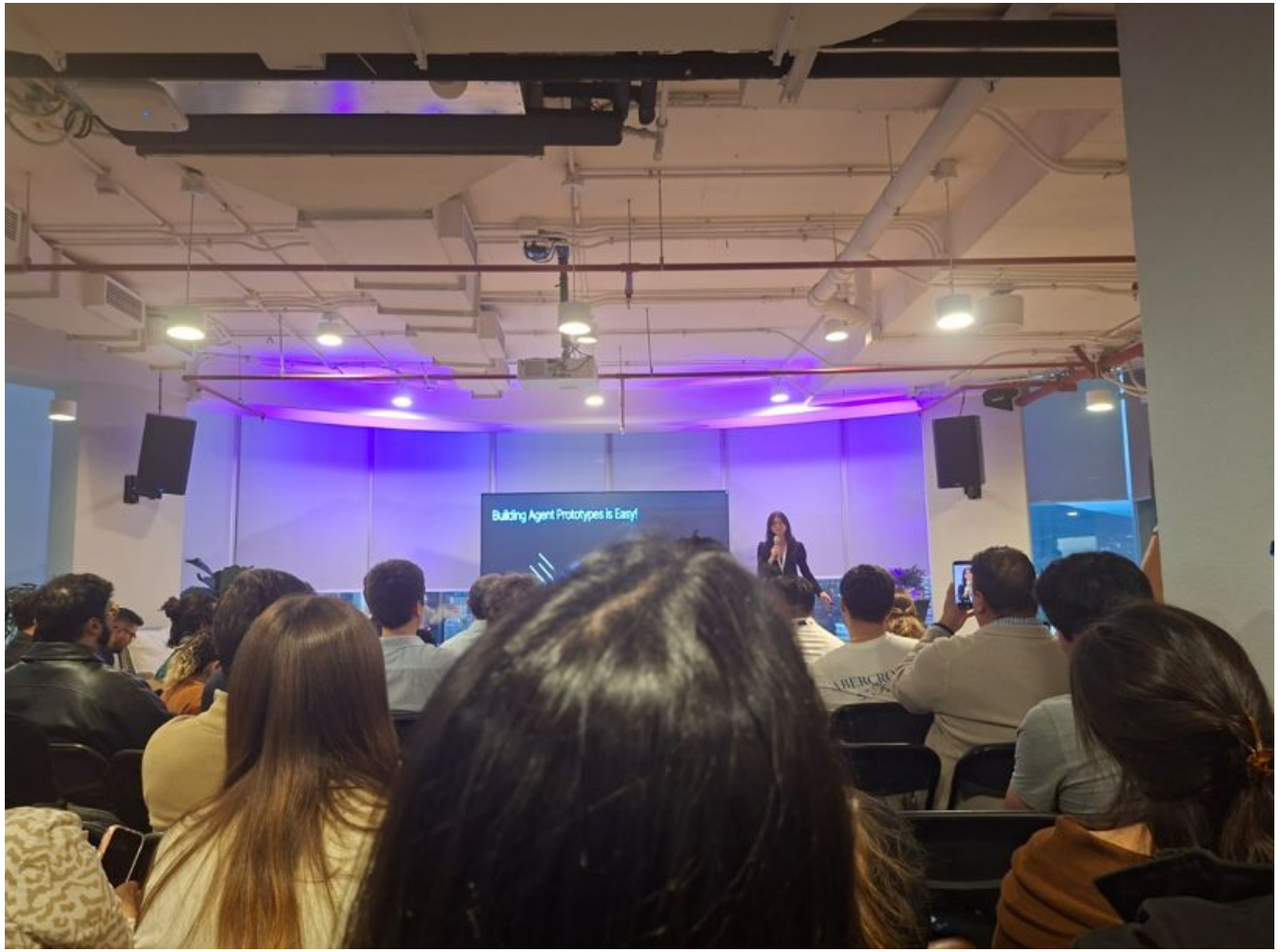
Ingeniería de soluciones con AI

Semestre 2026-2

Docente:

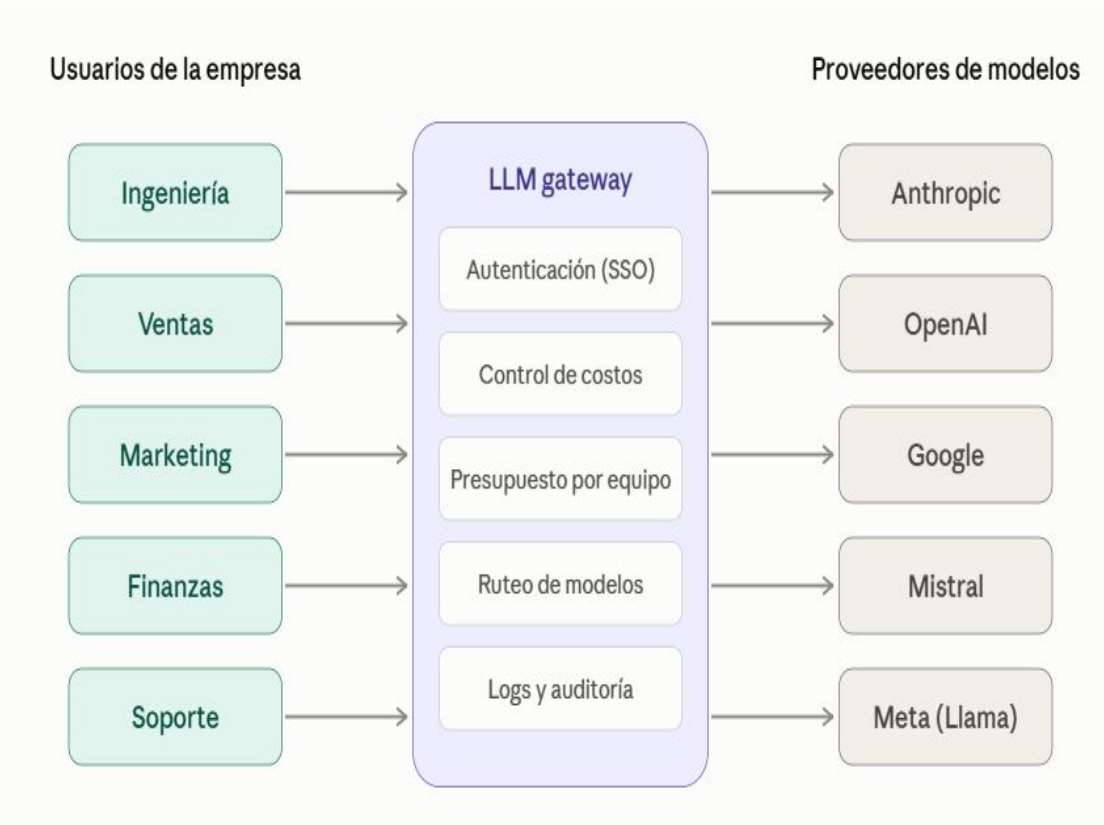
Francisco Macaya

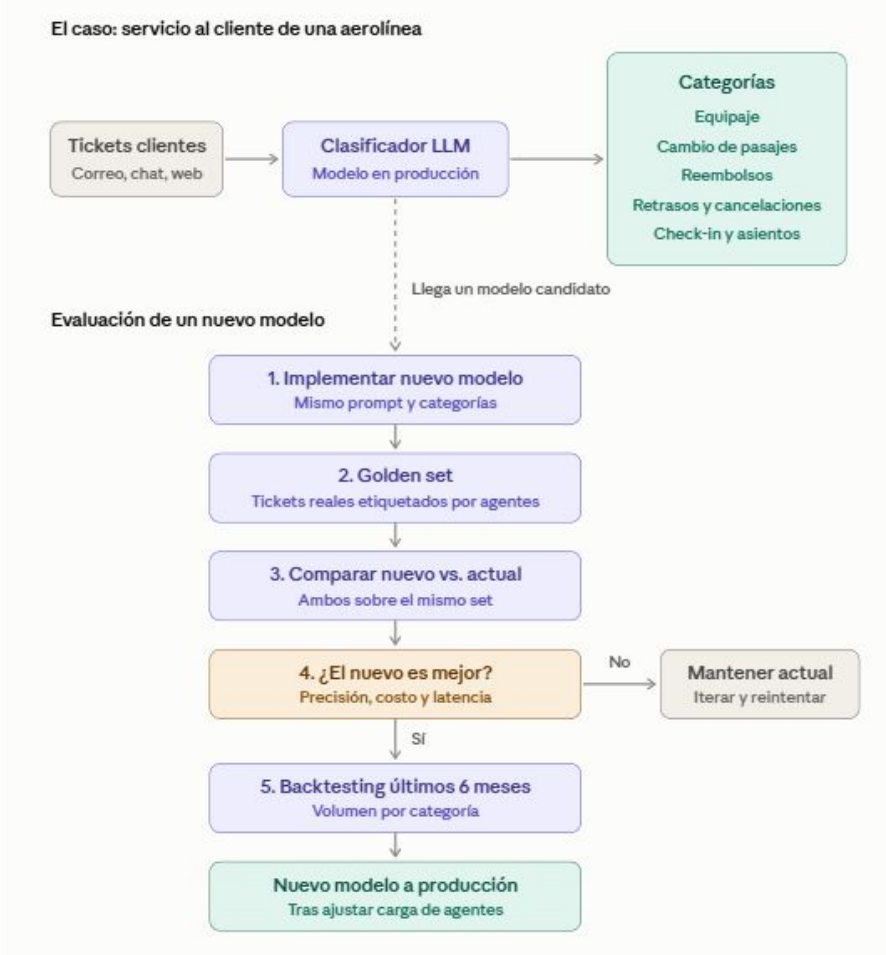
Noticias: *Impulse II by LATAM Airlines, LangChain & NeuralWorks*



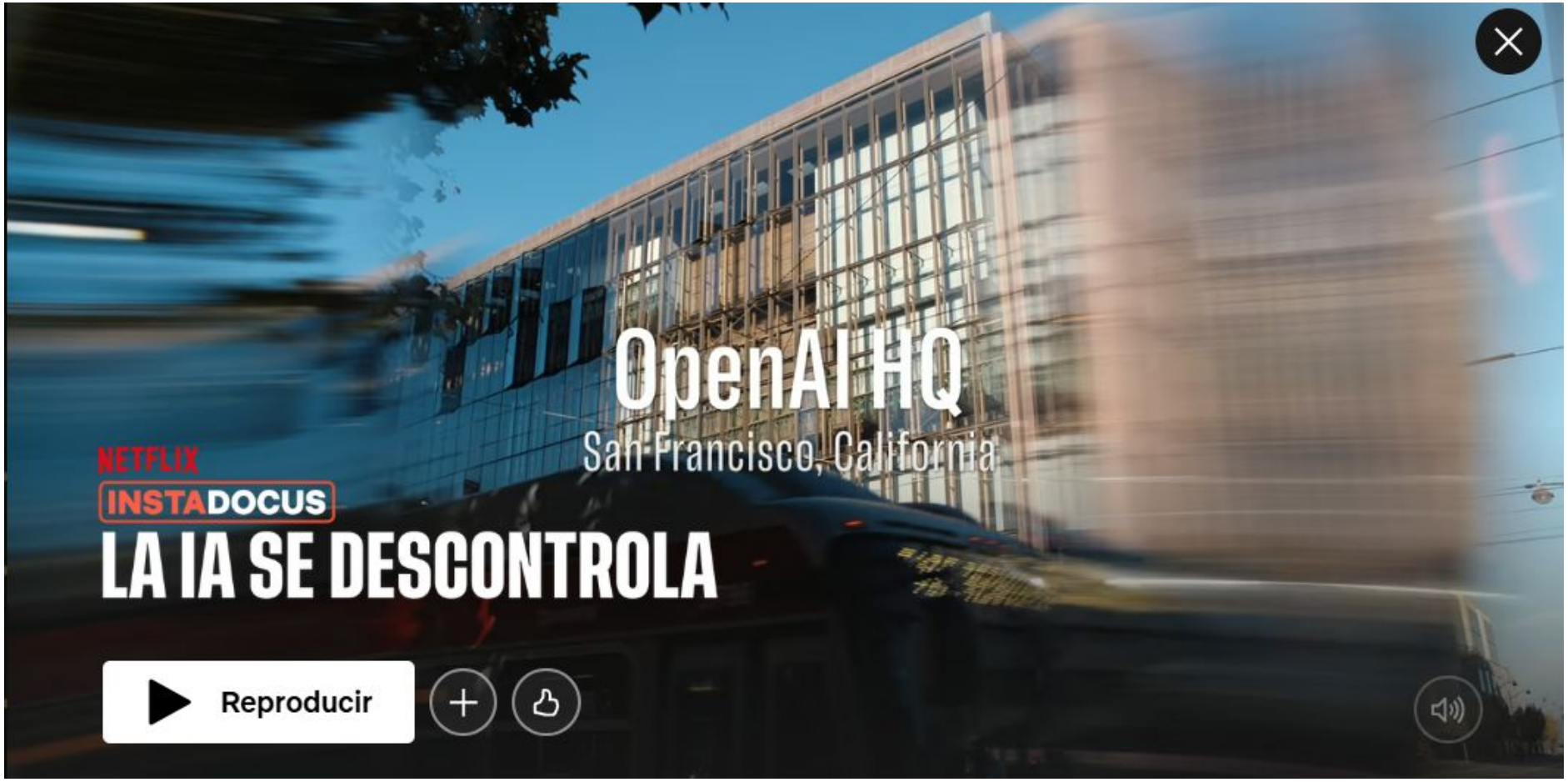
Nota:
https://www.langchain.com/blog/agent-observability-needs-feedback-to-power-learning

Noticias: *Dos caso de uso de LATAM Airlines.*



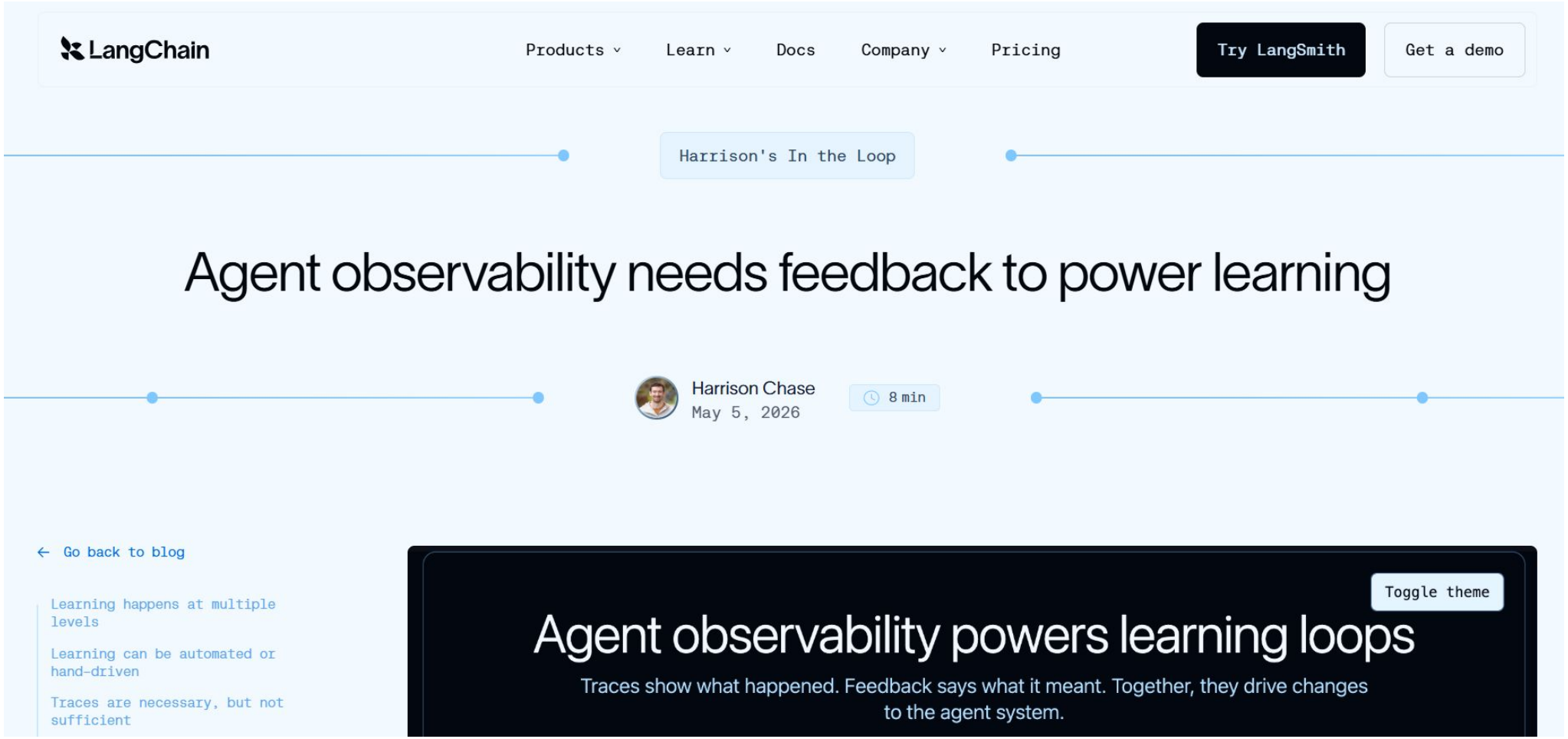


Noticias: *InstaDocus, Netflix.*



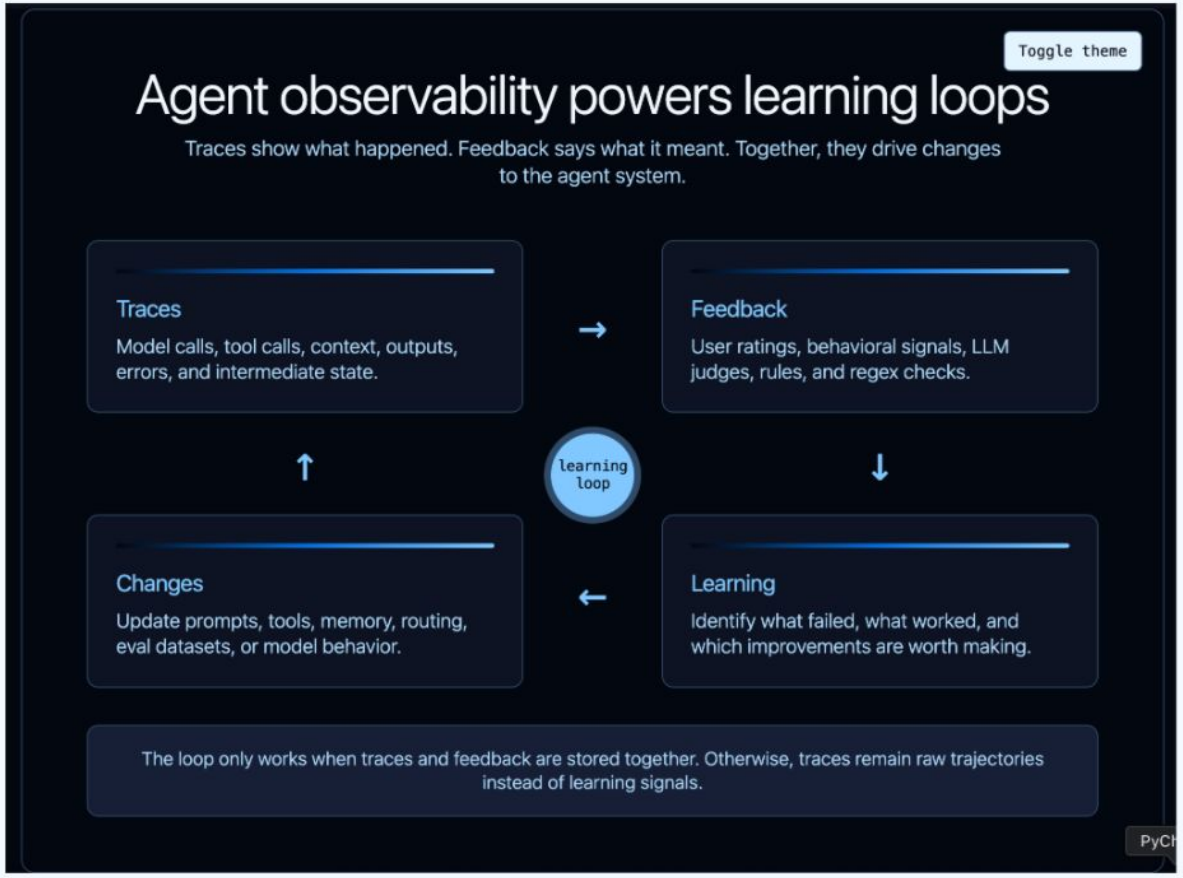
Nota: *https://www.netflix.com/title/82772145*

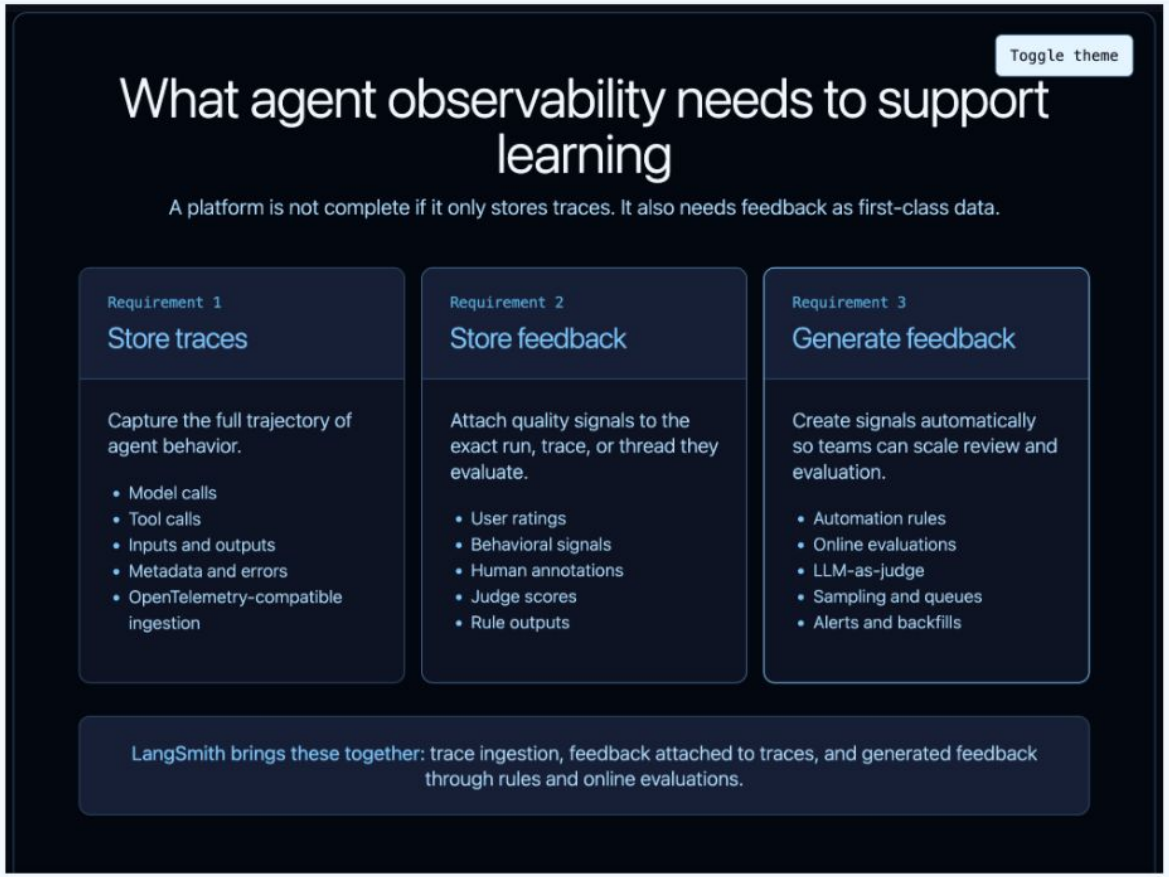
Noticias: *Agentes need feedback to work well.*



Nota: *https://www.langchain.com/blog/agent-observability-needs-feedback-to-power-learning*

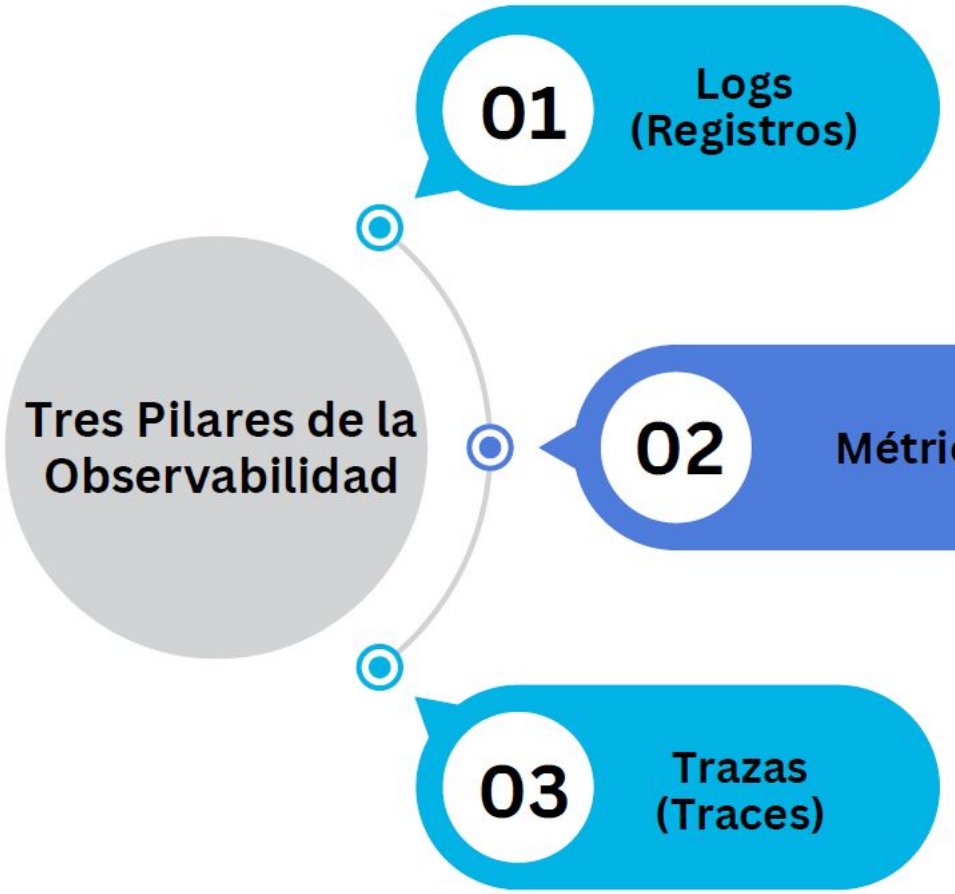
Observabilidad *no es solo detectar fallos, sino que mejorarlos.*





Tres "Stones": ***Logs, Metricas e Traces(Trazas).***

DuocUC

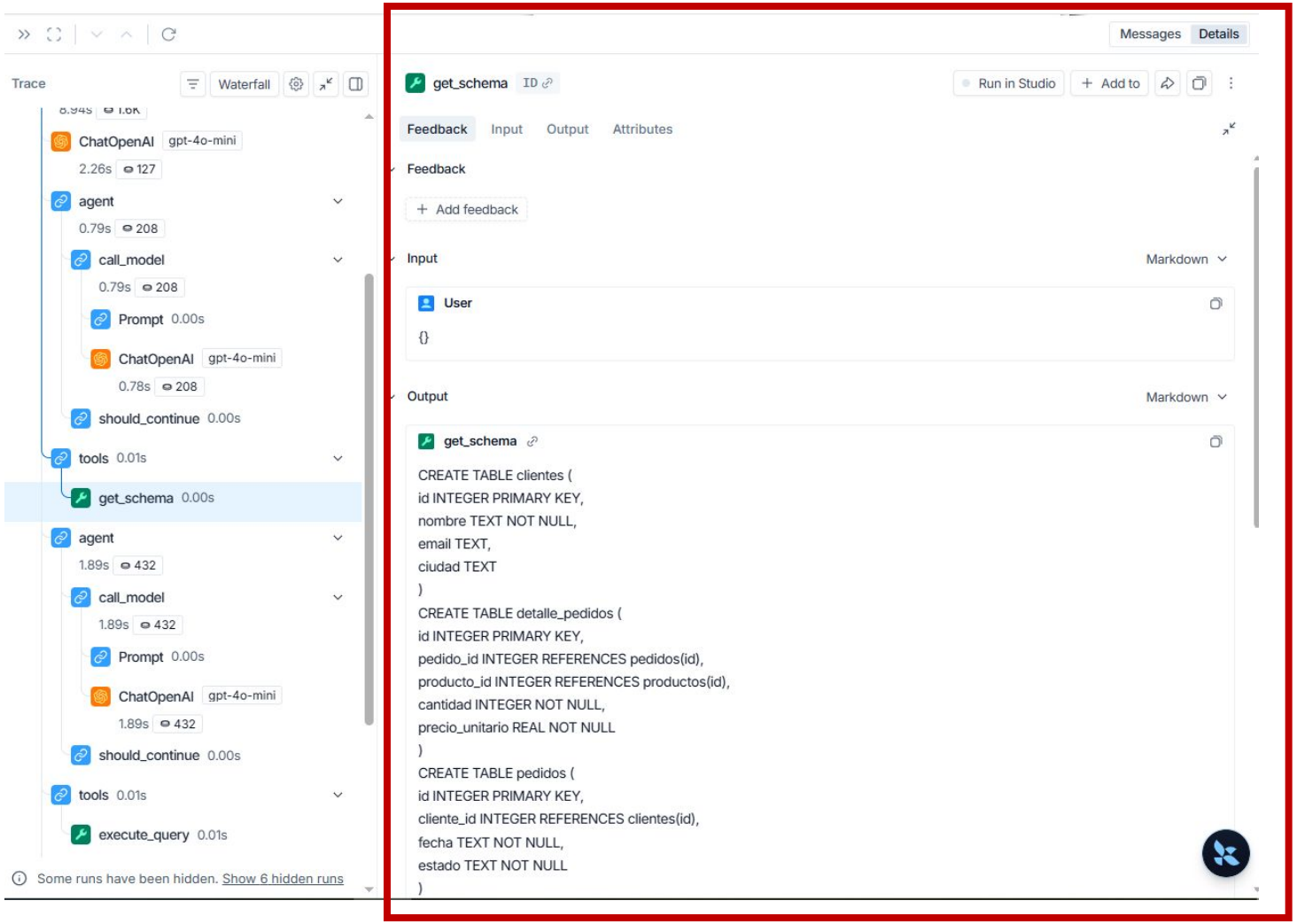


Eventos discretos (prompts, respuestas, uso de herramientas, pensamientos del agente).
Mejores Prácticas: Estructurados, contextuales, centralizados.

Valores numéricos agregados a lo largo del tiempo (latencia, uso de tokens, tasa de éxito).
Mejores Prácticas: Etiquetado, agregación, alertas.

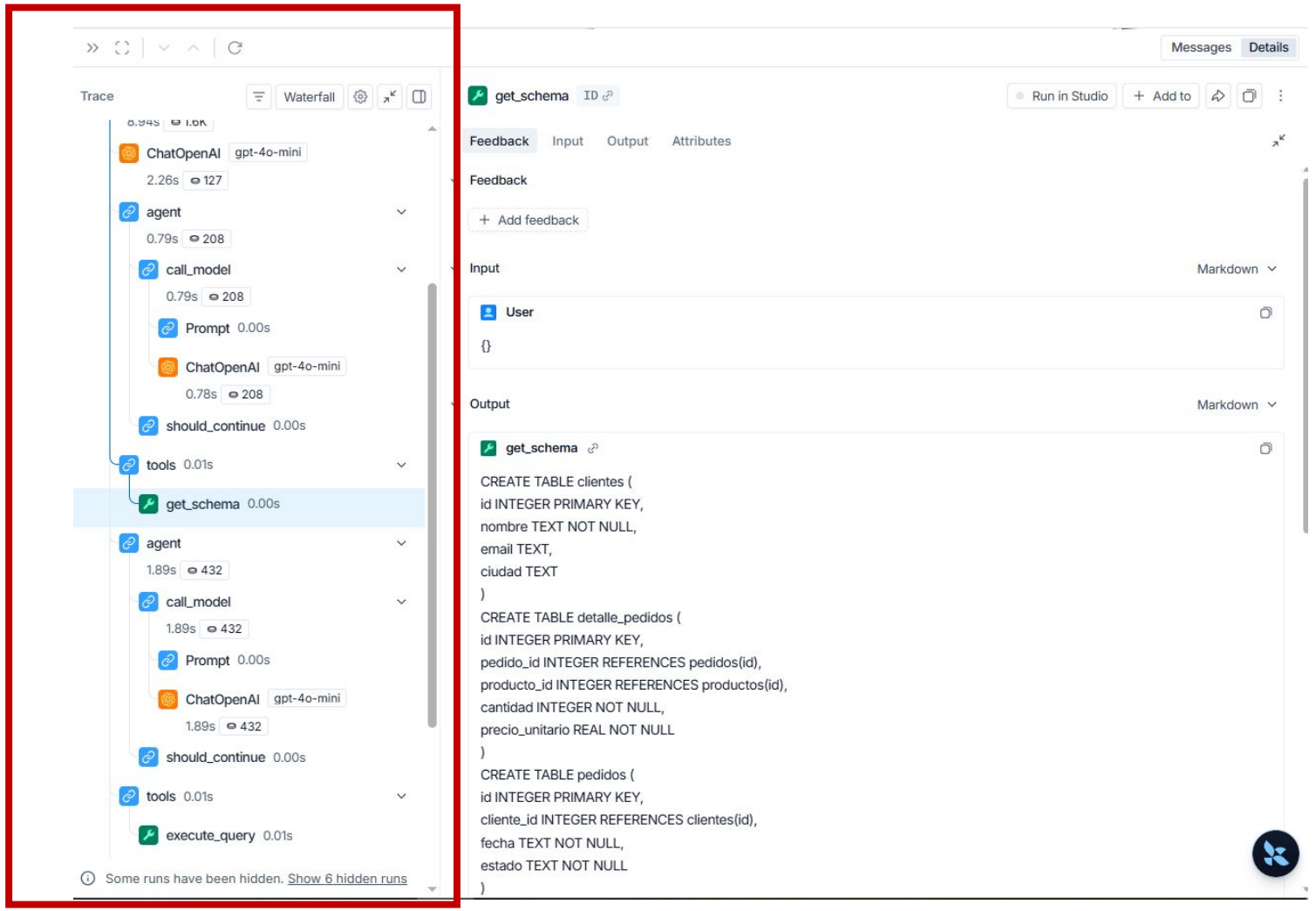
El camino completo de una solicitud a través del sistema, mostrando la secuencia de operaciones.
Mejores Prácticas: Instrumentación, visualización, correlación.

Logs: *Eventos discretos, uso de tools, etc.*



Un llamado a una tool es un log, una acción que hay que monitorear y trackear.

Traces: *El camino completo.*

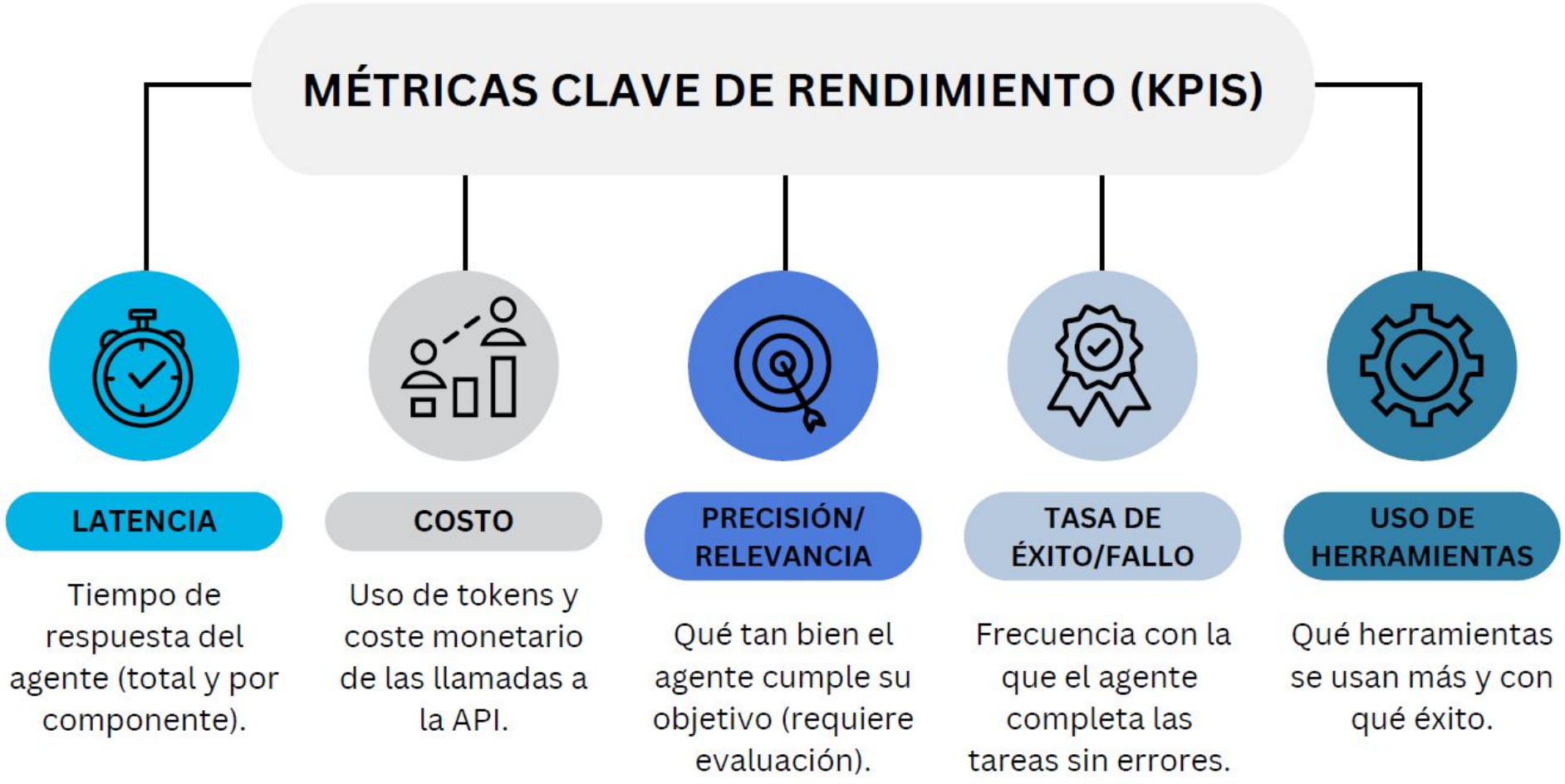


Usuario*: ¿Cuáles fueron las ventas del 2025?*

Todo el camino es el trace.

Asistente: *Las ventas fueron $250.000.000 en el 2025.*

Métricas: *Datos cuantificables del modelo.*



Tool Call Accuracy: *¿Llamo a las tools correctamente?*

Tool call Accuracy

`ToolCallAccuracy` measures how accurately an LLM agent invokes tools compared to expected tool calls. It evaluates both the sequence of tool calls and the accuracy of their arguments. This metric is particularly useful for validating that agents call the right tools with the right parameters in multi-step workflows.

The metric requires `user_input` (conversation messages) and `reference_tool_calls` (expected tool calls). It returns a score between 0 and 1, where higher values indicate better performance.

Key Features

Two Evaluation Modes:

1. **Strict Order (default)**: Tool calls must match exactly in sequence
   - Use for: Sequential workflows where order matters
   - Example: Must search before filtering results
2. **Flexible Order**: Tool calls can be in any order
   - Use for: Parallel operations where order doesn't matter
   - Example: Fetching weather for multiple cities simultaneously

Scoring:

- Evaluates sequence alignment (correct tools in correct order)
- Evaluates argument accuracy (correct parameters for each tool)
- Final score = (argument accuracy) × (sequence aligned ? 1 : 0)

```python
import asyncio
from ragas.metrics.collections import ToolCallAccuracy
from ragas.messages import AIMessage, HumanMessage, ToolCall

async def evaluate_tool_call_accuracy():
    # Define the conversation with tool calls
    user_input = [
        HumanMessage(content="What's the weather like in New York right now?"),
        AIMessage(
            content="The current temperature in New York is 75°F and it's partly cloudy.",
            tool_calls=[ToolCall(name="weather_check", args={"location": "New York"})],
        ),
        HumanMessage(content="Can you translate that to Celsius?"),
        AIMessage(
            content="Let me convert that to Celsius for you.",
            tool_calls=[
                ToolCall(
                    name="temperature_conversion", args={"temperature_fahrenheit": 75}
                )
            ],
        ),
    ]

    # Define expected tool calls
    reference_tool_calls = [
        ToolCall(name="weather_check", args={"location": "New York"}),
        ToolCall(name="temperature_conversion", args={"temperature_fahrenheit": 75}),
    ]

    # Evaluate
    metric = ToolCallAccuracy()
    result = await metric.ascore(
        user_input=user_input,
        reference_tool_calls=reference_tool_calls,
    )
    print(f"Tool Call Accuracy: {result.value}")

if __name__ == "__main__":
    asyncio.run(evaluate_tool_call_accuracy())
```

Agent Goal Accuracy: *¿Logro el objetivo?*

Agent Goal Accuracy

Agent goal accuracy is a metric that can be used to evaluate the performance of the LLM in identifying and achieving the goals of the user. This is a binary metric, with 1 indicating that the AI has achieved the goal and 0 indicating that the AI has not achieved the goal.

With Reference

`AgentGoalAccuracyWithReference` evaluates whether the agent achieved the user's goal by comparing the workflow's end state against a provided reference outcome. The reference represents the expected/ideal outcome.

```python
import asyncio
from openai import AsyncOpenAI
from ragas.llms.base import llm_factory
from ragas.metrics.collections import AgentGoalAccuracyWithReference
from ragas.messages import AIMessage, HumanMessage, ToolCall, ToolMessage


async def evaluate_agent_goal_accuracy_with_reference():
    # Setup LLM
    client = AsyncOpenAI()
    llm = llm_factory("gpt-4o-mini", client=client)

    user_input = [
        HumanMessage(
            content="Hey, book a table at the nearest best Chinese restaurant for 8:00pm"
        ),
        AIMessage(
            content="Sure, let me find the best options for you.",
            tool_calls=[
                ToolCall(
                    name="restaurant_search",
                    args={"cuisine": "Chinese", "time": "8:00pm"},
                )
            ],
        ),
        ToolMessage(
            content="Found a few options: 1. Golden Dragon, 2. Jade Palace"
        ),
        AIMessage(
            content="I found some great options: Golden Dragon and Jade Palace. Which one
        ),
        HumanMessage(content="Let's go with Golden Dragon."),
        AIMessage(
            content="Great choice! I'll book a table for 8:00pm at Golden Dragon.",
            tool_calls=[
                ToolCall(
                    name="restaurant_book",
                    args={"name": "Golden Dragon", "time": "8:00pm"},
                )
            ],
        ),
        ToolMessage(content="Table booked at Golden Dragon for 8:00pm."),
        AIMessage(
            content="Your table at Golden Dragon is booked for 8:00pm. Enjoy your meal!"
        ),
        HumanMessage(content="thanks"),
    ]

    metric = AgentGoalAccuracyWithReference(llm=llm)
    result = await metric.ascore(
        user_input=user_input,
        reference="Table booked at one of the chinese restaurants at 8 pm",
    )
    print(f"Agent Goal Accuracy: {result.value}")

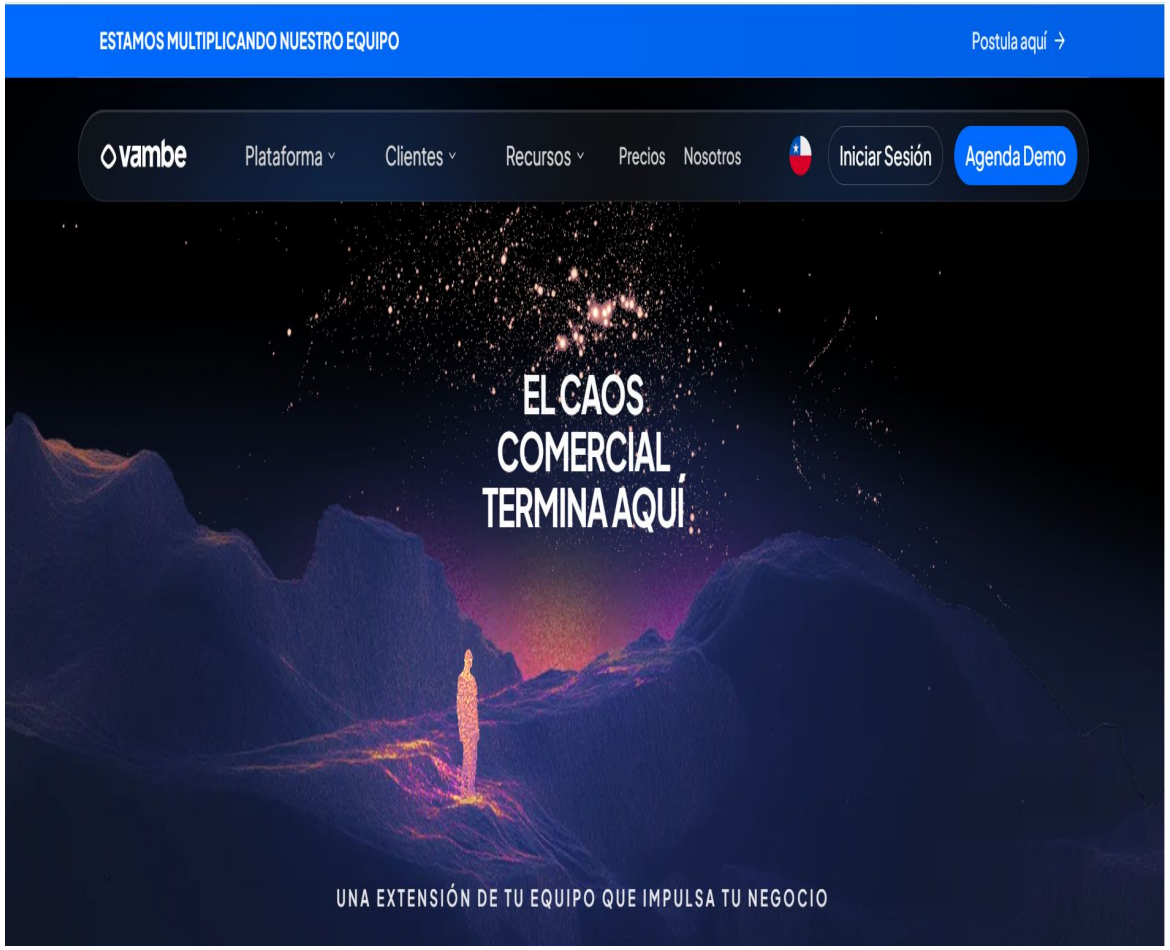

if __name__ == "__main__":
    asyncio.run(evaluate_agent_goal_accuracy_with_reference())
```

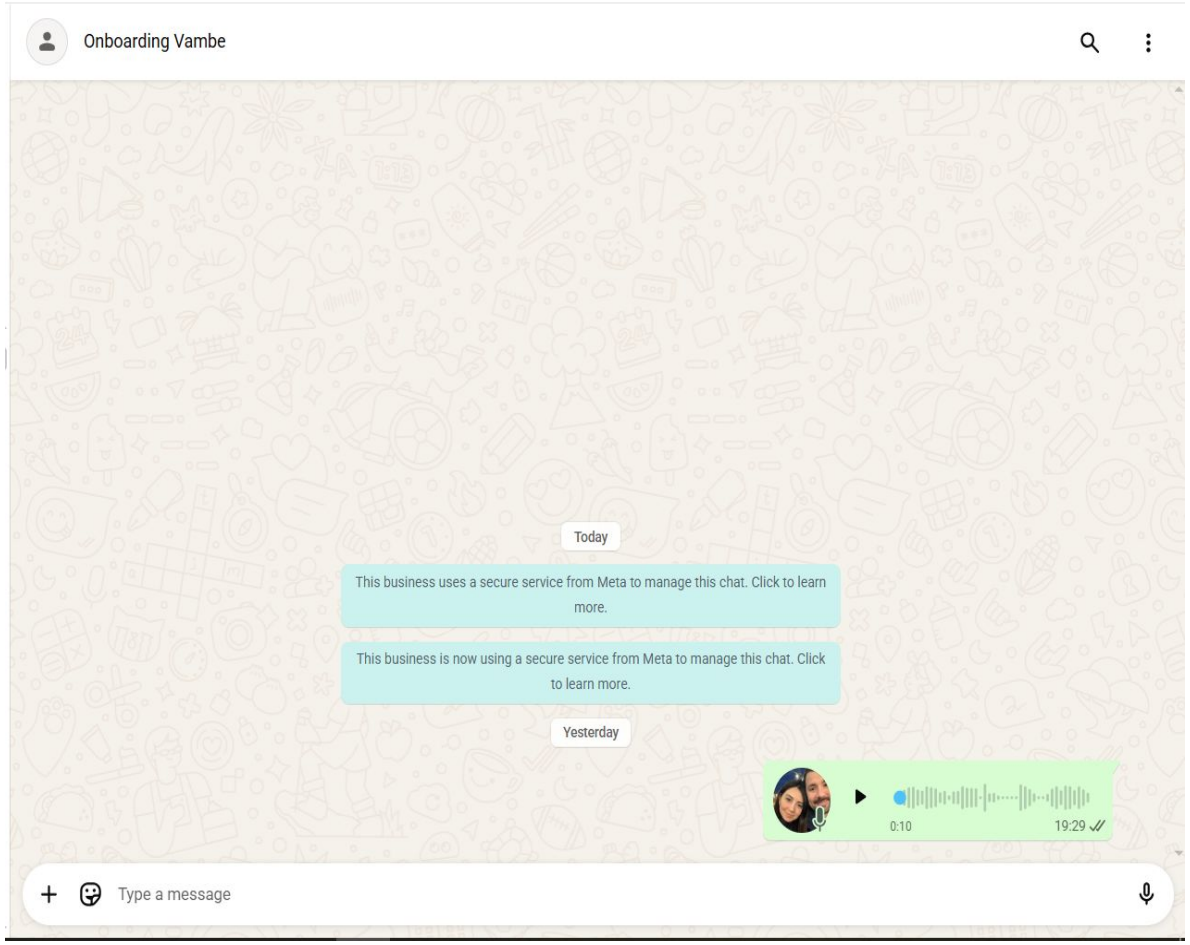
¿Qué necesito?: *El problema de los datos.*



Tienes que construir un dataset, pero no será suficiente.
Tendrás que llevarlo a producción en un ***marcha blanca.***

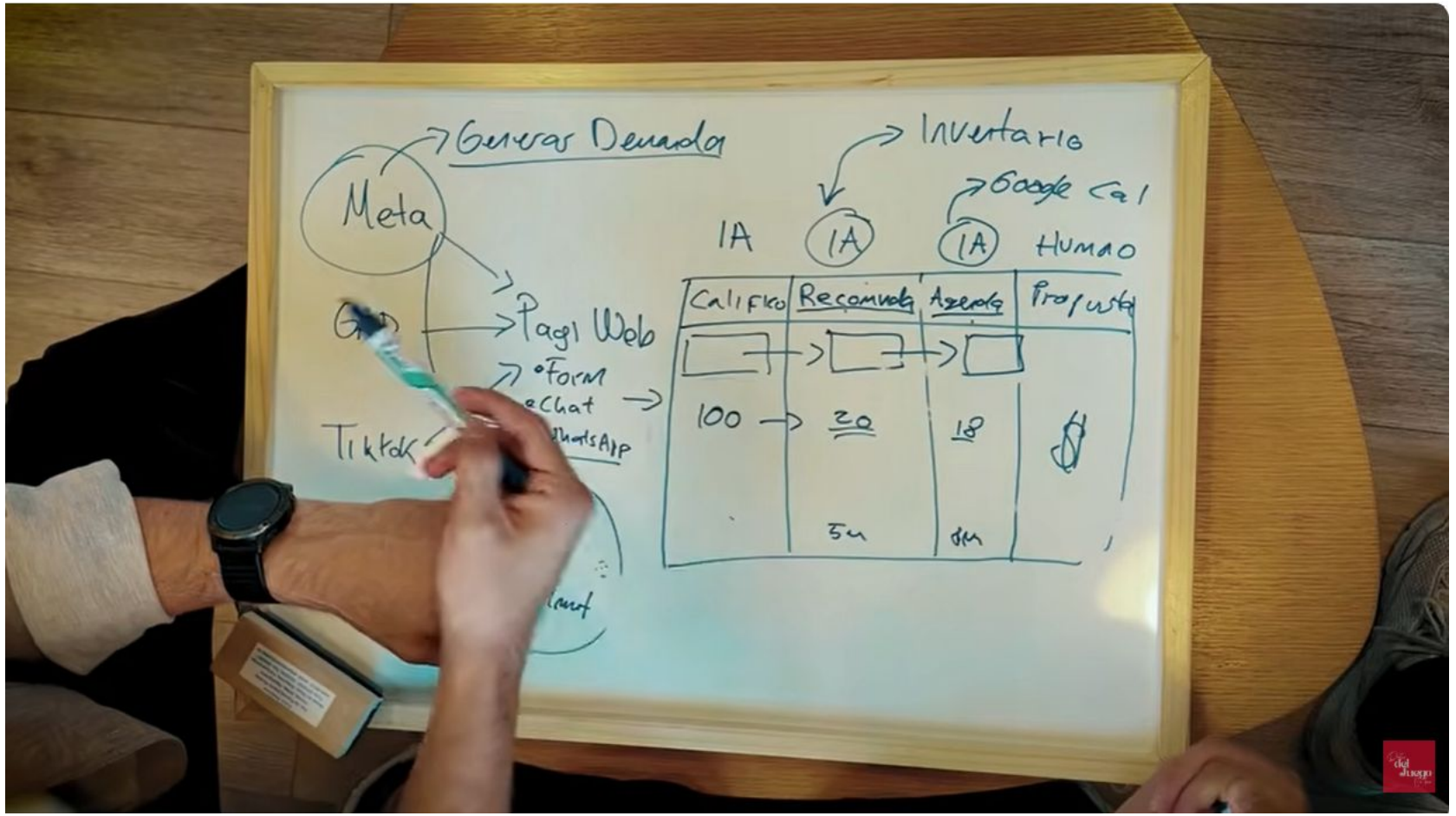
Vambe: *Start-up chilena de asistente de AI.*





Se implementa un agente, se "entrena" con personas del equipo, luego se da a otras personas (no clientes aun), y luego se libera a clientes.

Vambe: *Desde prospectos a conversión.*



https://www.youtube.com/watch?v=FfA2h-ZUG5Q